Understanding uniget, OCI, Artifacts, and Package Definitions

This walkthrough is designed to make the entire ecosystem click conceptually.

Big Picture

Most package systems look like this:

```
Package Manager
    ↓
Reads package definitions/metadata
    ↓
Finds artifacts/packages
    ↓
Downloads them from repositories/registries
    ↓
Extracts files onto system
```

Examples:

Package Manager	Definitions	Artifact	Repository
pacman	package metadata/PKGBUILD	.pkg.tar.zst	Arch repos
apt	package metadata	.deb	Debian repos
pip	package metadata	wheel/tar.gz	PyPI
docker	manifest/config	OCI image	Docker Hub
uniget	manifest/definition	OCI artifact	OCI registry

What is an Artifact?

An artifact is simply:

> A distributable built package.

Examples:

```
ffuf binary
ffuf.tar.gz
python wheel
Docker image
Arch package
```

Artifacts usually contain:

- binaries
- libraries
- configs
- metadata
- install scripts
- checksums

What is a Repository/Registry?

A repository stores artifacts.

Examples:

```
Arch repo -> stores .pkg.tar.zst files
PyPI -> stores Python wheels
Docker Hub -> stores OCI artifacts/images
GitHub Releases -> stores release binaries
```

Then Where Are Definitions Stored?

Excellent question.

Definitions/metadata may be stored:

- inside the artifact
- alongside the artifact
- in a separate index/database
- in git repositories
- in registry metadata

Different ecosystems do this differently.

Arch Example

PKGBUILD

The PKGBUILD exists separately:

```
PKGBUILD
    ↓
makepkg builds package
    ↓
.pkg.tar.zst artifact produced
```

Then inside the final package:

```
metadata also exists internally
```

So there are:

- build definitions
- package metadata

Both are different things.

Docker/OCI Example

Dockerfile

```
FROM ubuntu
RUN apt update
RUN apt install -y nginx
```

This is a build definition.

Then Docker builds:

```
OCI image artifact
```

Inside the OCI artifact are:

- filesystem layers
- manifests
- hashes
- config metadata

Important Distinction

There are usually TWO layers:

1. Build Definition

Example:

- PKGBUILD
- Dockerfile
- build.yaml
- manifest.json

Purpose:

```
How to create package
```

---

2. Package Metadata

Stored with artifact.

Purpose:

```
How to install/use package
```

OCI Is a Packaging + Distribution Standard

OCI defines:

- artifact structure
- manifests

- metadata layout
- layer format
- registry APIs
- hashes/checksums

OCI itself does NOT mean:

```
container execution
```

It mainly means:

```
standardized artifact format + registry ecosystem
```

Practical Walkthrough: Installing uniget

On Arch/Linux:

```
curl -sLf https://gitlab.com/uniget-org/cli/-/releases/permalink/latest/
downloads/uniget_Linux_$(uname -m).tar.gz
| sudo tar -xzC /usr/local/bin uniget
```

Verify:

```
uniget version
```

Search tools:

```
uniget search ffuf
```

Install a tool:

```
uniget install ffuf
```

Now check:

```
which ffuf
ffuf -V
```

Notice:

- ffuf runs directly on your host
- no container runs
- uniget simply downloaded/extracted the binary

Conceptual Custom Manifest Example

The exact uniget format may differ, but conceptually a definition might look like:

```
name: demo-tool
version: 1.0.0
artifact:
  type: tar.gz
  url: https://example.com/demo-tool.tar.gz
binary:
  name: demo-tool
  path: ./demo-tool
```

Conceptually this tells the package manager:

```
What package is this?
Where is artifact?
Which binary should be installed?
```

Very Important Systems Insight

Most modern infrastructure ecosystems are built around these abstractions:

```
files
    ↓
artifacts/packages
    ↓
registries/repositories
```

```
    ↓
package managers/installers
```

Once you understand this, suddenly:

- Docker
- OCI
- package managers
- CI/CD
- registries
- Kubernetes artifacts
- SBOM systems
- software supply chain security

all start feeling related instead of random disconnected technologies.

Suggested Experiment

After installing uniget:

Compare installation methods

pacman

```
sudo pacman -S ripgrep
```

go install

```
go install github.com/projectdiscovery/httpx/cmd/httpx@latest
```

uniget

```
uniget install httpx
```

Then inspect:

```
which httpx
ls -l $(which httpx)
```

You will start seeing:

```
All package managers ultimately move files around.
```

The rest is metadata, distribution, integrity, and ecosystem design.